

Questions You Can Expect From Judges

HACK TO THE FUTURE · DEMO DAY PREP

Why we're sharing these

A lot of you wrote in your check-ins that the Q&A is the part you're most worried about. We hear you. We're not going to share the rubric — we want to see how you actually think under questioning, not your rehearsed answers to the exact criteria. But we *do* want you ready.

These ten questions aren't the exact ones the judges will ask. They're the *shape* of the questions you'll get. If you can answer questions like these naturally, the actual ones won't catch you off-guard.

A request: practice these by **saying** the answers out loud, not writing them down. Judges can hear when an answer was memorised. The goal is to build muscle memory for the *thinking*, not the words.

About what you built

1. Walk me through your project in 90 seconds.

This is almost always the opener. Aim for: the problem in one sentence, who has it in one sentence, what you built in one sentence, the one technical bet you made in one sentence, and one specific user moment that shows it working. If you go past 90 seconds, you're padding.

2. Show me the part you're proudest of. Now show me the part you're least confident about.

This question rewards honesty, not perfection. Knowing exactly where your project is weak is a strength signal — it tells the judges you've stress-tested your own work. The wrong answer is "everything is solid." Nothing built in four weeks is solid everywhere.

3. What's the single most important technical decision you made, and why that over the alternative?

Have an answer ready for whatever your big architectural fork was — RAG vs fine-tuning, on-device vs cloud, structured vs unstructured retrieval, sync vs async, monolith vs services, whatever. The judges care less about *which* path you picked than about whether you picked it deliberately. "I used GPT-4 because it works" is a weak answer. "I picked GPT-4 over an open-source model because latency mattered more than cost on the demo budget, and we can swap later" is a real one.

About your user

4. Describe one specific user — give them a name, an age, the city they live in, what they were doing yesterday — for whom this product matters.

"Indian users" is not an answer. "Salaried 32-year-old in Pune who has ₹2L in her salary account, has been checking FD rates for three weeks but won't book because she's heard small finance banks fail" is an answer. Specificity tells the judges you actually thought about a real human, not a market segment.

5. When does your product *fail* the user? Walk me through that moment.

Every product has failure modes. The strong answer is one where you describe a specific scenario — "if the user types in code-mixed Hinglish with English numbers and Hindi nouns, our intent classifier drops to 60% accuracy, here's why" — and what you'd do about it next. If you say "it doesn't fail," you've actually failed the question.

About your build process

6. Take me through your worst day of building this. What broke, what did you try, what finally worked?

This is a story question. Tell the story. Include the dead ends, include the moment you almost gave up, include the fix that came at 2 AM. Builders who can talk about how they actually got unstuck — including the bad ideas they tried first — are the builders we want to hire. The wrong answer is "everything went smoothly."

7. What did you cut from your scope, and why?

If you didn't cut anything, you probably didn't ship. Builders who cut things ship things. Have one or two specific cuts ready to talk about, with the reasoning behind each. "I cut the multilingual TTS because it added 4 seconds of latency that destroyed the user experience and the topic only required text response" is the right shape.

8. If you had another month on this, what's the first thing you'd build next, and why that over other things?

The judges aren't looking for a feature list. They're looking for whether you have a *roadmap shape* in your head — a sense of priorities, dependencies, and what would unlock the most value next. "I'd add live partner-bank rates because right now I'm using sample data, and a real rate-fetch turns this from a demo into something a user would actually rely on" is better than "I'd add more features."

About the bigger picture

9. Who else is trying to solve this problem, and where do you think they're getting it wrong?

Don't dismiss competitors as "not real competitors." If your project is FD-related, you're competing with Jar, Wint, Stable Money, Fixed.com, INDmoney's FD section. Name them. Be specific about what they do well *and* where the gap is. Honest competitive awareness signals maturity. Pretending you have no competition signals the opposite.

10. Forget I'm a judge — pitch this to me as if I'm the user. Thirty seconds, no jargon.

This is a register-switching test. Drop every acronym, every framework name, every reference to your tech stack. Talk to a real person. If you can't explain what your product does to your grandmother in plain language, it doesn't matter how clever the architecture is.

Things to avoid in Q&A

- **Memorised answers.** Judges hear them immediately.
 - **Defensiveness.** Every challenge is information. "I hadn't considered that, let me think out loud" is a great response.
 - **Faking certainty.** "I don't know yet, here's how I'd find out" is stronger than a wrong confident answer.
 - **Selling.** This isn't a pitch. The judges aren't a VC firm. They're trying to figure out how you think.
 - **Filler.** "So basically what I built is essentially..." — every word like this lowers the signal in your answer. Pause instead of filling.
-

One last thing

The Q&A is shorter than you think — usually four minutes. That's room for three or four real exchanges, not ten. So don't try to memorise answers to every possible question. Get good at the *shape* of these ten, and you'll be ready for whatever they actually ask.

Saturday is going to go faster than you expect. Walk in steady, listen to the question, and answer the one you were asked — not the one you wish you'd been asked.

You've got this.